Keys and shelves：讲稿（中英对照）

页码对应 **Keys_and_shelves_evidence_pages.pdf**。

第 1 页 · Keys and shelves（开场）

这一页用一个比喻回答我们的核心问题：模型是“书在书架上，但拿不出来”，还是“钥匙不行，根本取不到”？书架是模型存的知识，橙色那本书是这篇论文的作者，钥匙是论文标题，也就是取书的线索；右边的气泡是模型最后说出的名字，说得晚，也说得不确定。下面我分四步论证：先证明书在不在，再看钥匙什么时候转、把信息送到哪里，然后排除“书卡住了”的解释，最后看书放在哪个架子上。

This slide answers our core question with one metaphor: is the book on the shelf but stuck, or is the key failing to get it out? The shelves are what the model stores, the orange book is this paper's authors, and the key is the title, the cue used to retrieve them. The speech bubble is the name the model finally says: late, and uncertain. I'll argue in four steps: first, whether the book is there; second, when the key turns and where the information goes; third, ruling out "the book is stuck"; and fourth, which shelf it sits on.

第 2 页 · How we read the model（打分）

先讲我们怎么打分。我们把真作者名放进回答里，看模型在第 l 层对每个名字 token 给的排名，记分 s = log10(1/名次)：排第 1 得 0 分，排第 10 得 −1，排第 100 得 −2。一个名字的分数是它所有 token 的平均。关键的量是标题特异信号：同一份作者名单，配真标题和配假标题各跑一遍，信号等于真标题的分数减去 5 个假标题的平均分。假标题借自别的论文，年份等其他部分保持不变。信号大于 0，说明是“这个标题”让模型更偏向它的真作者，而不是名字本身就好猜。

First, how we score. We place the true author names in the answer and look at the rank the model gives each name token at layer l. The score is s = log10(1/rank): rank 1 scores 0, rank 10 scores −1, rank 100 scores −2. A name's score is the average over its tokens. The key quantity is the title-specific signal: we run the same author list once with the true title and once with false titles borrowed from other papers, keeping everything else, including the year, unchanged. The signal is the true-title score minus the mean over five false titles. A positive signal means it is this title that pushes the model toward its real authors, not that the names are easy to guess.

第 3–4 页 · 分组

我们用多选题把论文分成两组：target 是模型自己写不对作者、但多选题四次都认得出真名单的论文，一共 4,621 篇；control 是多选题也认不出的，2,220 篇。比较两组时，我们在每个引用档内部配对，排除“论文越有名、分数越高”的影响。

We split papers into two groups using a multiple-choice test. Target papers are those where the model cannot write the authors itself, yet picks the true author list in all four orderings: 4,621 papers. Control papers are those it cannot recognise even in multiple choice: 2,220 papers. When comparing the groups, we match within citation bins, so the difference is not just "more famous papers score higher."

第 5 页 · Can we trust the numbers?

这页说明数字的可靠性，包括快速前向和逐 token 计算的一致性检查。需要坦白的一点是：一致性容差是在看到差距之后才从 0.25 放宽到 0.5 的，这一点写在局限里；而且这次放宽确实用到了：Stage 10 主轮 25 个（读出 × 层）组合的最大差 0.367，只有按 0.5 才通过。原定 0.25 时，Stage 10 主轮一致性检查（12 个上下文、52 个 token 位置）的 25 个 lens × 层组合中有 10 个超出 0.25，最大差 0.367（351_stage10/reports/parity_before_tol05_amendment.json）；3-3 在 7 篇论文的 23,280 行上按 0.25 计 99.81% 在容差内。这一量级与 3-3 查明的 bf16 舍入随前向长度变化（最大约 0.46）一致，但容差是在看到数据后放宽到 0.5 的，属于事后修改。批量大小本身造成的组级 E1 恢复比例差最大 0.0148（L40），单 token 的 log10(1/名次) 差 p99 最高 0.22（补充 12 第 2-4 项，381_stage13/reports/b2_e1noise.md）。

This slide covers reliability, including parity checks between the fast forward pass and token-by-token scoring. One thing I want to be upfront about: the parity tolerance was relaxed from 0.25 to 0.5 after we saw the gaps, and this is listed as a limitation; the relaxation was actually used: the Stage 10 main round (25 readout × layer pairs, max difference 0.367) passes only at 0.5. With the original 0.25, 10 of the 25 lens × layer pairs of the Stage 10 main-round parity check (12 contexts, 52 token positions) exceeded 0.25, max 0.367 (parity_before_tol05_amendment.json); in 3-3, 99.81% of 23,280 rows (7 papers) were within 0.25. This size matches the bf16 rounding that changes with forward length found in 3-3 (up to about 0.46), but the tolerance was relaxed to 0.5 after seeing the data, a post-hoc change.

第 6 页 · ① 书在书架上吗？在（输出层）

第一个问题：模型到底有没有存这篇论文的作者？在最后一层 L62，真标题比假标题高约 0.44。也就是说，换掉标题，模型对真作者的偏好就明显下降，这个偏好确实来自这篇论文。

First question: does the model store this paper's authors at all? At the final layer, L62, the true title beats false titles by about 0.44. Swap the title out and the model's preference for the real authors drops clearly, so that preference comes from this specific paper.

第 7–8 页 · ① 书在书架上吗？在（probe）

第二个证据来自 probe。我们取标题最后一个 token 在第 l 层的内部状态，用 ridge 回归训练一个线性映射，让它预测模型自己在冒号处最后一层的状态，因为这个状态直接决定模型说出的第一个名字。然后把预测出的状态变成对 4,314 个名字 token 的打分，看每位作者第一个名字 token 排第几；这个 token 大多是首字母。真标题减的是 false_0 一个假标题。为了防止泄漏，我们按作者团队切分，同一团队的论文全在训练集或全在测试集；凡是提示里的标题在训练中出现过的论文都剔除，最后剩 1,863 篇，E4b 信号只用其中 target + control 的 1,395 篇。结果从 L24 起 +0.05 到 +0.08，target 比 control 高 +0.03 到 +0.08。原版标准化用了全部论文（含测试折），改成只在训练折内标准化；两种标准化差别最大 0.0007，L40 为 +0.061、L56 为 +0.075，结论不变。对照实验里把答案换成别的论文的作者，得分约为 0。所以信息存在，而且是这篇论文特有的，只是信号不大，而且主要来自首字母这种信息量很少的 token。补充 9（答辩后新增，分析计划在看结果前写定）把这个 probe 扩展到每位作者的全部名字 token 并按角色分开：L40 姓的第一个 token +0.071 [0.058, 0.085]（预注册版 V0：只减 false_0，目标 + 对照 1,395 篇，剔除标题复用，团队聚类 95% CI；381_stage13/reports/e4b_alltok.md），高于首字母 +0.044 [0.030, 0.057]（1,590 篇）；名的第一个 token 最强（L40 +0.124、L56 +0.163（1,706 篇））。原 E4b 只打分第一个 token（多半是首字母），所以低估了标题位置携带的名字信息。

The second piece of evidence is a probe. We take the hidden state of the last title token at layer l and fit a ridge-regression linear map that predicts the model's own final-layer state at the colon, since that state directly determines the first name the model says. We then turn the predicted state into scores over 4,314 name tokens and look at the rank of each author's first name token, which is usually an initial. The true title is compared with false_0 only. To prevent leakage, we split by author team, so a team's papers are all in training or all in test, drop every paper whose prompt contains a title seen in training, That leaves 1,863 papers, of which the E4b signal uses the 1,395 target and control papers. The probe reads +0.05 to +0.08 from L24 onward, and target exceeds control by +0.03 to +0.08; refitting the standardisation on training folds only changes the values by at most 0.0007 (+0.061 at L40, +0.075 at L56). When the answer is swapped for another paper's authors, the score is about zero. So the information is there and specific to this paper, though the signal is small and comes mostly from initials, which carry little information.

第 9–10 页 · ② 钥匙是标题，在 L44–52 读进名字位置

第二个问题：标题里的信息在哪一层、送到了哪个位置？我们用替换实验。真标题跑一遍、假标题跑一遍，在第 l 层把真标题那遍某些位置的内部状态复制到假标题那遍，其余照常往后算。恢复率等于（替换后的分数减假标题分数）除以（真标题分数减假标题分数），0 是没恢复，1 是完全恢复。我们先把所有论文的分子、分母各自平均再相除，避免真假差距接近 0 的论文把比值拉爆。替换标题之后的整段提示，L48 恢复 0.82，L52 起约等于 1。Stage 12 进一步一次只换一部分：只换回答里的名字位置，L48 是 0.61、L52 是 0.85；只换冒号最多 0.17；其他单个位置都不超过 0.03。我们还做了阻断：从第 l 层起不让标题之后的所有位置（包括冒号和名字位置）看标题，从 L52 起阻断几乎没有损失。所以标题信息在 L44 到 52 进入名字位置，到 L52 已经全部拿到。另外要说明，第 10 页的标题写的是 Stage 11 的原始结论 L24–L50，Stage 12 把它细化成了 L44–52。

Second question: at which layer, and into which position, does the title's information move? We use activation patching. We run the prompt once with the true title and once with a false title, then at layer l copy the hidden states at selected positions from the true run into the false run and let the rest compute normally. Recovery is (patched score minus false score) divided by (true score minus false score): 0 means nothing restored, 1 means fully restored. We average numerators and denominators across papers before dividing, so papers with a near-zero gap cannot blow up the ratio. Patching the whole prompt after the title recovers 0.82 at L48 and about 1.0 from L52. Stage 12 then patches one part at a time: the name positions alone recover 0.61 at L48 and 0.85 at L52; the colon alone at most 0.17; every other single position at most 0.03. We also ran knockout, blocking every position after the title, including the colon and name positions, from attending to the title from layer l onward; from L52 on, it costs almost nothing. So the title's information enters the name positions at L44 to 52 and is fully there by L52. One note: slide 10's title still shows the original Stage 11 conclusion, L24 to L50; Stage 12 refined it to L44 to 52.

第 11–12 页 · ③ 书在但卡住了？不成立

第三个问题：会不会是模型在中间层已经清楚知道作者，只是被什么挡住了说不出？如果是这样，中间层 target 组的信号应该已经明显高于 control。我们在每个引用档内算两组之差再加权平均，置信区间按作者团队分组 bootstrap。结果中间层的差约为 0，到 L57–61 才升起来，输出层达到 +0.21。所以不是知识清楚地放着却出不来，而是取出的过程本身就弱，而且晚。

Third question: could the model already know the authors clearly in the middle layers, but be blocked from saying them? If so, the target group's signal should already exceed control's in the middle layers. We compute the group difference within each citation bin, take a weighted average, and bootstrap confidence intervals by author team. The difference is about zero in the middle layers and only rises at L57 to 61, reaching +0.21 at the output. So it is not that clear knowledge is stored but cannot get out; the retrieval itself is weak, and late.

回到第 1 页 · ④ 哪个书架？两个都用，有先后

第四个问题：信息是靠注意力从标题搬过来的，还是靠 MLP 从记忆里调出来的？我们用同样的替换，但只换第 l 层名字位置上一个部件的输出。L48 注意力恢复 0.23，同层 MLP 只有 0.07；L52 和 L56 的 MLP 分别恢复 0.18 和 0.22，注意力只有 0.11 和 0.06；L8 到 L44 每个单独部件都不超过 0.03。所以是先由 L48 的注意力把标题信息读进来，再由 L52–56 的 MLP 加工。有两个限制：一次只换一层，信息分散在多层时会被低估，所以这些数是下限；MLP 那一步是不是在查记忆，这个实验还分不出来。

Fourth question: is the information carried from the title by attention, or pulled from memory by the MLPs? We use the same patching, but replace only one component's output at the name positions at layer l. At L48, attention recovers 0.23 while the MLP recovers only 0.07. At L52 and L56, the MLPs recover 0.18 and 0.22, while attention recovers only 0.11 and 0.06. From L8 to L44, no single component exceeds 0.03. So attention at L48 first brings the title in, and the MLPs at L52 to 56 then process it. Two limits: patching one layer at a time underestimates information spread across layers, so these are lower bounds; and this experiment cannot tell whether the MLP step is a memory lookup.

第 13–15 页 · Caveat：读出工具能看到什么

最后一个提醒：读出工具没找到，不等于信息不存在。按名次打分的 lens 只能看到“像词表里某个词”的信息；中间层可能存的是任务或论文身份，这种形式 lens 测不到。这三页展示了各种 lens 在中间层基本读不出名字，所有 lens 都到 L62 才跳起来。

A final caveat: a readout not finding something does not mean it is absent. A rank-based lens only sees information that looks like a vocabulary token. The middle layers may hold the task, or the paper's identity, in a form the lens cannot score. These three slides show that every lens reads almost nothing in the middle layers, and they all jump only at L62.

第 16 页 · Stage 13：三个假设

这就是 Stage 13 要回答的问题：名字出现之前，中间层到底在表示什么？我们列了三个假设。H1 是任务和格式，在冒号和名字位置成立：lens 在那里看到的是格式符号，而且换问法时冒号位置的状态移动是换论文的 3 到 5 倍（L4–L40；L48 约 2.2 倍，L56–L63 为 1.3–1.5 倍）（x1_similarity.md colon 表）。H2 是论文身份，只能说部分支持：标题位置能读出年份、引用数、领域，而且从 L16 就开始；把问年份时的标题状态换进问作者那遍，作者分数恢复得差不多。但 M3 按预注册要比 L4 的文字水平高才算数，结果并不比 L4 高，所以“换问法不变”可能只是因为三种问法的标题文字一样。H3 是作者已经在那里、只是不在词表方向上，证据不足：专门用作者提示拟合的 lens 也读不出；probe 从 L8 起能读出一点第一作者，但只靠标题文字的基线不弱于 Qwen，所以不能排除标题文字的解释。三个假设并不互斥：冒号和名字位置承载任务，标题位置能读出论文属性，名字本身直到 L57–61 都在 lens 里看不见。X3 已经跑完，结论是否定的：改写标题在 Qwen 中间层（L28–L40，最高 L36 +0.16）比用词相近的近邻 D 更接

近原标题，超出 L4 文字水平；行为上，改写保留了原标题对 D 约 90% 的第一作者偏好。但两项检验说明这不足以证明“论文身份”：第一，只懂语义、不认识任何作者的句子向量模型 all-mpnet，在用词真正匹配的子集上也有相当的优势；第二，认不认得出没有任何差别：行为上的 90% 在 target（0.90）和 control（0.91）之间一样，中间层的优势也一样（L36 上 target +0.158、control +0.160，各层两组之差都约为 0）。如果是认出了论文，target 应该更高。要主动说明一点：按补充 4 预注册的字面判据（G′ > 0 且超出 L4，调参集找到、测试集确认），X3 其实判为“存在论文身份信号”；文本模型基线和 target/control 拆分是看到结果后加的检验，我们据此把解读降为“语义表示”。两者都如实报告，这一步让结论更保守，而不是更乐观。所以 X3 只说明 Qwen 中间层表示了标题的意思，而不只是用词，分不清是认出论文还是懂得意思。Qwen 特别的只是时间：优势出现在中间层，后层又消失。总结起来，M3、X3、X4 这三个专门用来区分“认出论文”和“读懂文字或意思”的实验，都没能证明是认出论文。

This is the question Stage 13 asks: before a name appears, what do the middle layers represent? We set out three hypotheses. H1, task and format, holds at the colon and name positions: the lens sees format tokens there, and the colon's state moves three to five times more when the question changes than when the paper changes (L4–L40; 2.2× at L48, 1.3–1.5× at L56–L63)(x1_similarity.md, colon table). H2, paper identity, is only partly supported: year, citations and field are readable at the title position from L16, and patching in the title state from the year question restores author scores about as well. But under our pre-registration M3 only counts if it beats the L4 text level, and it does not, so "unchanged across questions" may simply reflect identical title text in the three prompts. H3, authors already present but off the vocabulary directions, has insufficient evidence: even a lens fitted on author prompts reads nothing, and a probe reads a little of the first author from L8, but a title-text-only baseline is not weaker than Qwen, so title wording cannot be ruled out. The three are not mutually exclusive: the colon and name positions carry the task, the title position carries readable paper attributes, and the name itself stays invisible to the lens until L57–61. X3 has finished, and the answer is negative: in Qwen's middle layers (L28–L40, peaking at +0.16 at L36) the paraphrased title is closer to the original than a word-similar neighbour D, beyond the L4 text level; behaviourally the paraphrase keeps about 90% of the original title's first-author advantage over D. Two checks show this is not enough to establish paper identity. First, all-mpnet, a sentence-embedding model that understands meaning but knows no authors, shows a comparable advantage on the subset where word overlap is truly matched. Second, recognition makes no difference: the behavioural 90% is the same for target (0.90) and control (0.91), and so is the middle-layer advantage (at L36, target +0.158 and control +0.160; the group difference is about 0 at every layer). If the model were recognising the paper, target should be higher. One point to state up front: under the literal pre-registered criterion in addendum 4 (G′ > 0 and above L4, found on the tune split and confirmed on test), X3 counts as showing a paper-identity signal; the text-model baseline and the target/control split were added after seeing the results, and on that basis we downgrade the reading to "semantic representation". We report both; this makes the conclusion more conservative, not less. So X3 shows that Qwen's middle layers represent the title's meaning, not just its words, but cannot separate recognising the paper from understanding the meaning. What is distinctive about Qwen is only the timing: the advantage appears in the middle layers and fades later. Taken together, M3, X3 and X4, the three experiments designed to separate recognition from wording or meaning, all fail to show recognition.

可能的追问（Q&A）

Q1：③ 用的是读出工具，可 Caveat 又说读出工具会漏看，这不矛盾吗？ ③ 只能说明，在读出工具能看到的那种形式上，中间层两组没有差别。中间层是否以别的形式存着两组的差别，正是 Stage 13 在测的；probe 在标题位置已经看到 target 高于 control。

Q1: Point 3 relies on a readout, yet the caveat says readouts can miss things. Isn't that a contradiction? Point 3 only shows no group difference in the form a readout can see. Whether the middle layers hold the difference in another form is exactly what Stage 13 tests, and the probe at the title position already shows target above control.

Q2：替换实验只用一个假标题，结果会不会碰巧？ 原来只用了 false_0。X6 用另外 4 个假标题补做了替换，复现检查与 Stage 11 原数据完全一致，结果是：5 个假标题平均，L48 恢复 0.82（target 0.79、control 0.88），L52 恢复 0.995（target 0.99、control 1.01）；每个假标题和 false_0 的差在 L48 不超过 0.02、L52 不超过 0.01，预注册允许差到 0.10。所以时间点不是 false_0 碰巧造成的。

Q2: The patching used only one false title. Could that be luck? Originally it used only false_0. X6 repeated the patching with four more false titles, with a replication check that matched the Stage 11 data exactly. The result: averaged over five false titles, recovery is 0.82 at L48 (target 0.79, control 0.88) and 0.995 at L52 (target 0.99, control 1.01); each false title differs from false_0 by at most 0.02 at L48 and 0.01 at L52, against a pre-registered allowance of 0.10. So the timing is not an accident of false_0.

Q3：阻断会不会只是把模型搞乱了？ X7 对假标题那遍也做同样的阻断，用真标题的损失减去假标题的损失，扣掉“被搞乱”的部分，结果是：从 L0 起挡，论文特有的信号全部消失（约 1.0）；从 L52 起挡，target 只剩约 2%，control 约 5%（刚好超过预注册的 0.05），从 L54 起降到 2%，L56 起两组都是 0。所以扣掉“被挡本身的破坏”之后，结论不变：L52 前后标题就不再被需要，control 晚一两层。替换实验本身不会把模型带离正常状态，而它给出的时间点和阻断一致。

Q3: Could knockout just be breaking the model? X7 applies the same knockout to the false-title run and subtracts its loss, removing the damage from blocking itself. The result: blocking from L0 removes the paper-specific signal entirely (about 1.0); blocking from L52 leaves about 2% for target and about 5% for control (just over the pre-registered 0.05), falling to 2% from L54 and to 0 for both groups from L56. So after removing the damage of blocking itself the conclusion holds: the title is no longer needed around L52, one or two layers later for control. Patching does not push the model off its normal states, and it gives the same timing as knockout.

Q4：probe 为什么只用 1,863 篇？ 为了排除泄漏，我们剔除了所有提示里的标题在训练中出现过的论文。修复前后按原切法重跑，数字逐位复现，说明代码没变；在这批更干净的论文上，结论仍然成立。

Q4: Why does the probe use only 1,863 papers? To rule out leakage, we removed every paper whose prompt contains a title seen in training. Re-running the original split reproduced the published numbers exactly, so the code did not change, and the conclusion holds on this cleaner set.